

# Diplomatura en Inteligencia Artificial

---

Detección de objetos en  
imágenes

Esp. Ing. Juan Ignacio García  
Universidad Nacional de Cuyo

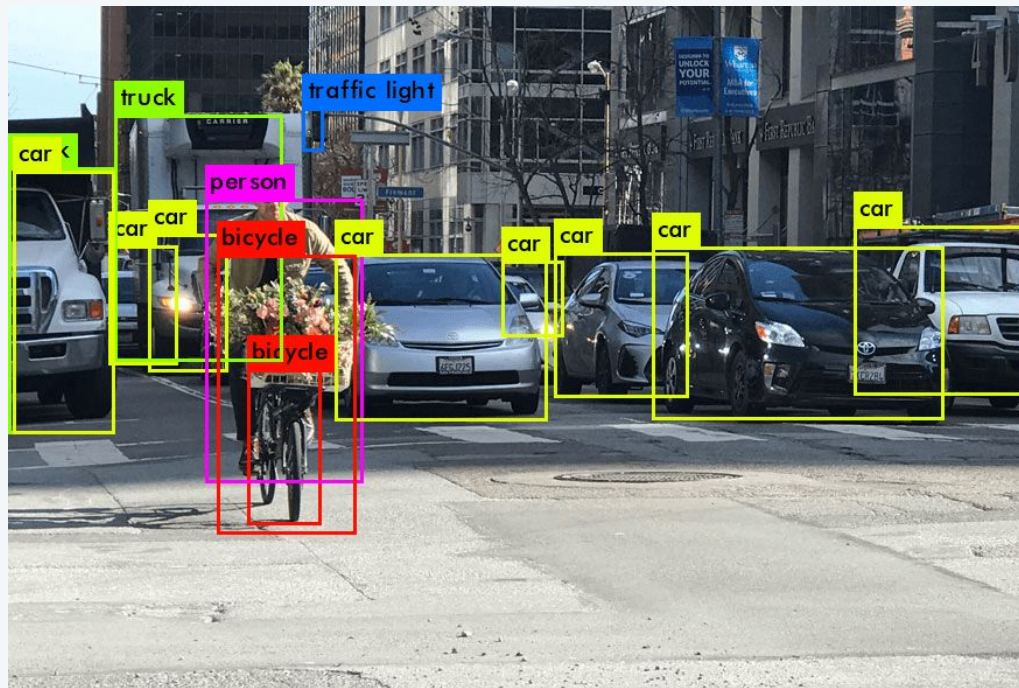


# Múltiples clasificaciones por imagen

2

Las arquitecturas vistas hasta ahora ayudan a detectar la **presencia** de un objeto en la imagen. Un problema distinto es identificar **dónde** están y **cuántas** veces aparecen.

Se modifican las estructuras para detectar la **clase** y la **ubicación** de cada objeto.

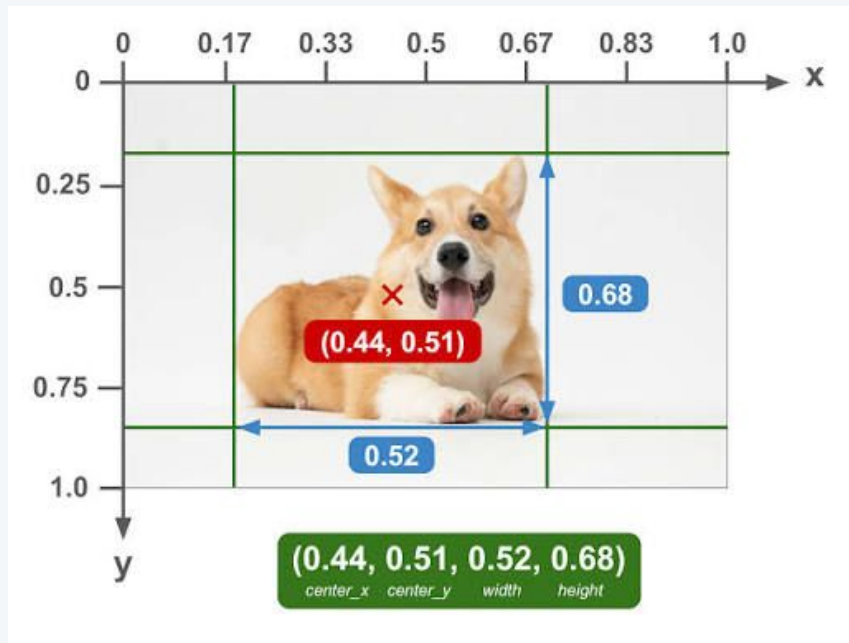


# Bounding boxes

3

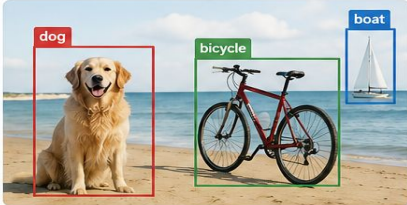
La herramienta utilizada para marcar la **posición** de un objeto dentro de una imagen es el bounding box.

Consiste en una capa más de datos que registra la **posición x** e **y**, el **ancho** y el **largo** de un rectángulo medido en píxeles. Adicionalmente indica la **clase** del objeto a detectar.



La información se registra manteniendo un **formato estandarizado**. Los formatos más comunes son:

### PASCAL VOC

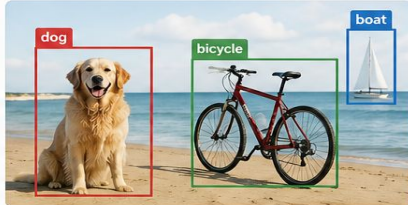


**XML** imagen.xml

```
<annotation>
  <object>
    <name>dog</name>
    <bndbox>
      <xmin>34</xmin>
      <ymin>45</ymin>
      <xmax>315</xmax>
      <ymax>420</ymax>
    </bndbox>
  </object>
  ...
</annotation>
```

■ dog  
■ bicycle  
■ boat

### YOLO



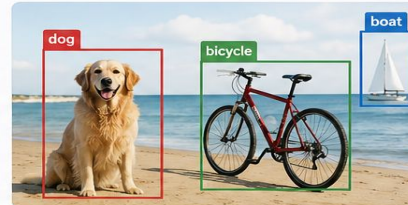
**TXT** imagen.txt

```
0 0.278 0.465 0.320 0.625
1 0.626 0.535 0.485 0.630
2 0.875 0.220 0.100 0.190
```

| class | x_center | y_center | width | height |
|-------|----------|----------|-------|--------|
| 0     | +        |          |       |        |
| 1     | ...      |          |       |        |
| 2     |          |          |       |        |

■ dog  
■ bicycle  
■ boat

### COCO



**JSON** imagen.json

```
{
  "images": [
    {
      "id": 1, "file_name": "imagen.jpg", ...
    }
  ],
  "annotations": [
    {
      "id": 1, "image_id": 1, "category_id": 1,
      "bbox": [34, 45, 281, 375], "area": ...
    },
    {
      "id": 2, "image_id": 1, "category_id": 2,
      "bbox": [420, 180, 310, 320], "area": ...
    },
    {
      "id": 3, "image_id": 1, "category_id": 3,
      "bbox": [820, 80, 80, 120], "area": ...
    }
  ],
  "categories": [
    {
      "id": 1, "name": "dog",
    },
    {
      "id": 2, "name": "bicycle",
    },
    {
      "id": 3, "name": "boat"
    }
  ]
}
```

■ dog (1)  
■ bicycle (2)  
■ boat (3)

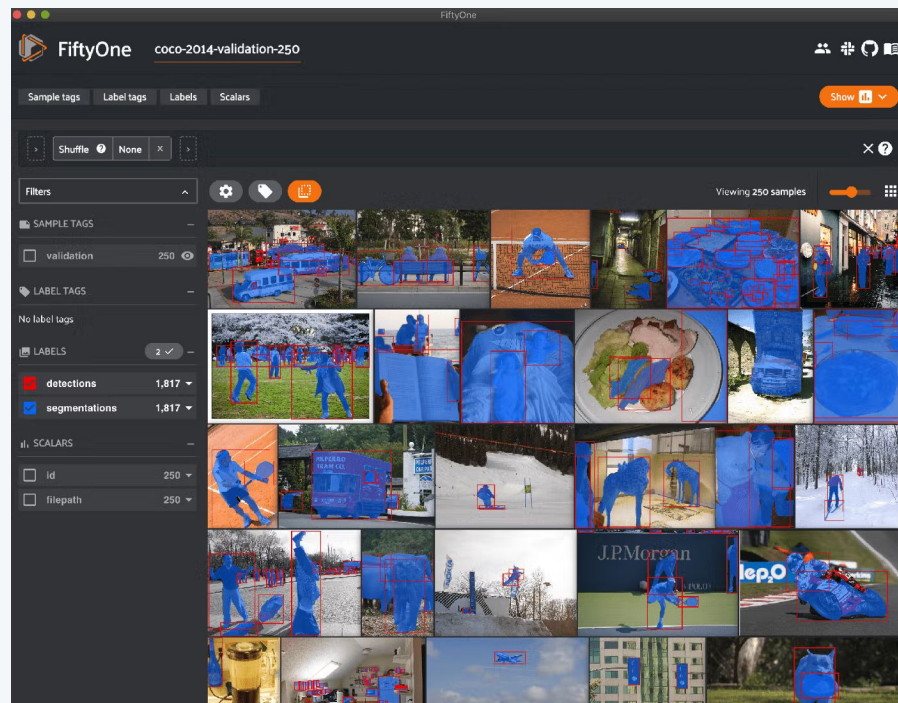
 1 archivo XML por imagen

 1 archivo TXT por imagen

 1 archivo JSON para todo el dataset

El formato COCO se impone sobre los demás porque:

- **Concentra** la información en un único archivo.
- Permite la **representación** de otros polígonos, descriptores y metadatos.
- Es fácil de indexar en el procesamiento de los modelos.



# Sliding window

6

Las redes ahora tienen una tarea doble:

- Regresión: detectar la **ubicación** de un objeto de interés.
- Clasificación: detectar la **clase** del objeto.

Todas las posiciones evaluadas por Sliding Window



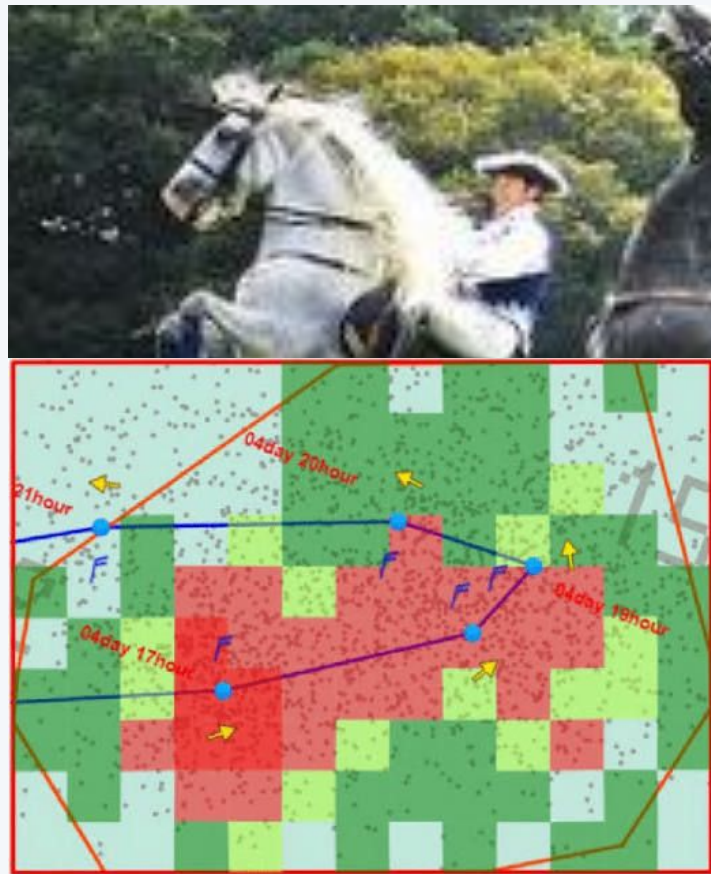
La primera solución que se diseña se llama “**sliding window**”, y consiste en usar las **redes CNN** sobre distintas zonas de la imagen hasta encontrar la coincidencia de **mayor probabilidad**.

El problema con esta solución es la **escalabilidad**. La solución intuitiva es usar pirámides, pero requiere **mucho procesamiento** sobre la misma imagen.



Para solucionarlo se crean las convolutional sliding windows, que generan un **feature map** de la imagen y aplica un clasificador que indica la **presencia** del objeto en cada posición.

Esto **reduce** la cantidad de procesos sobre la imagen y contribuye a la **invarianza** de la imagen.

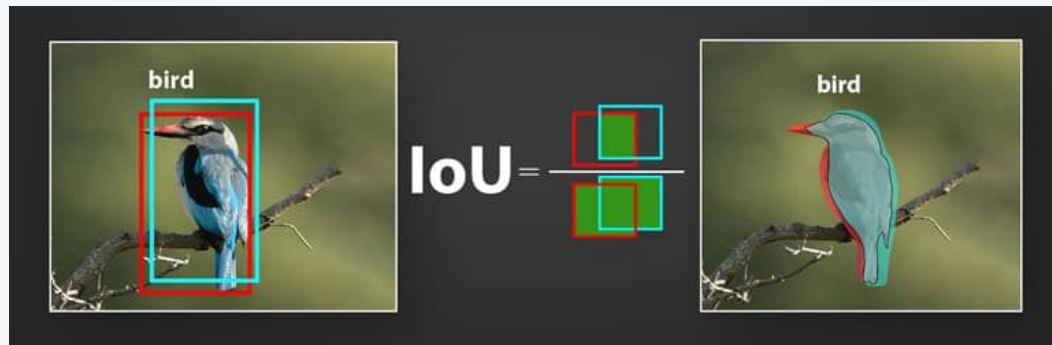


# Intersection over Union (IoU)

9

Para poder medir qué tan precisa es la **ubicación** predicha por el modelo, se utiliza el **coeficiente** que mide la **intersección** de las áreas de los bounding boxes sobre el área total de su **unión**.

Se definen **umbrales** a partir de los cuales se considera una medición válida o no.

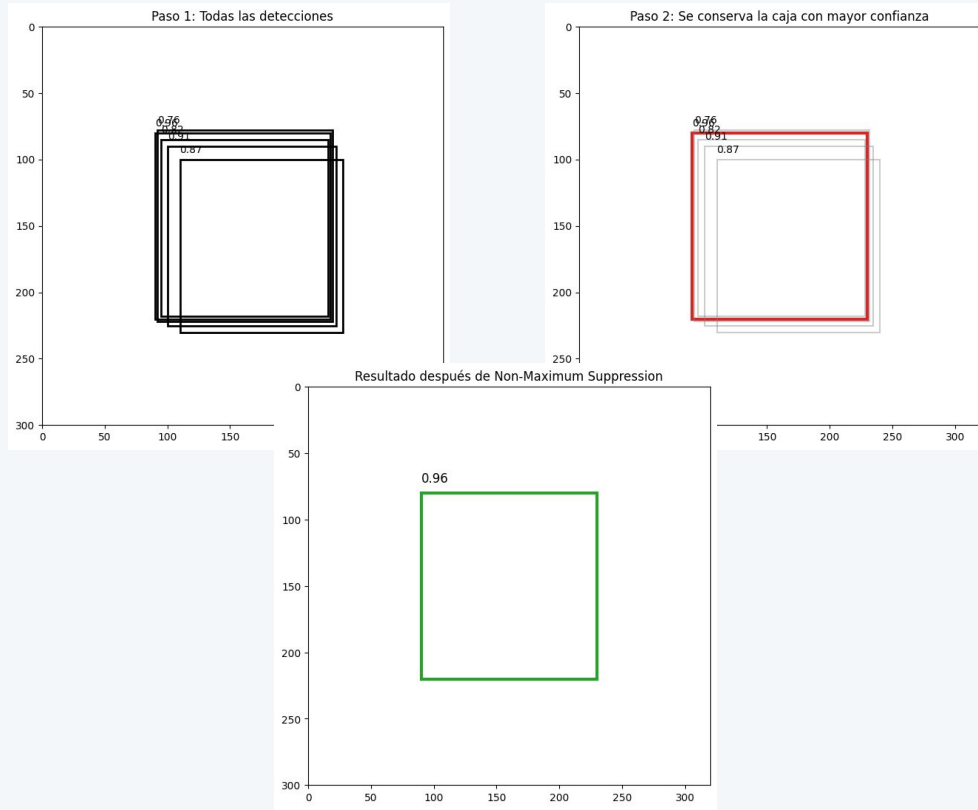


# Supresión de no-máximos (NMS)

10

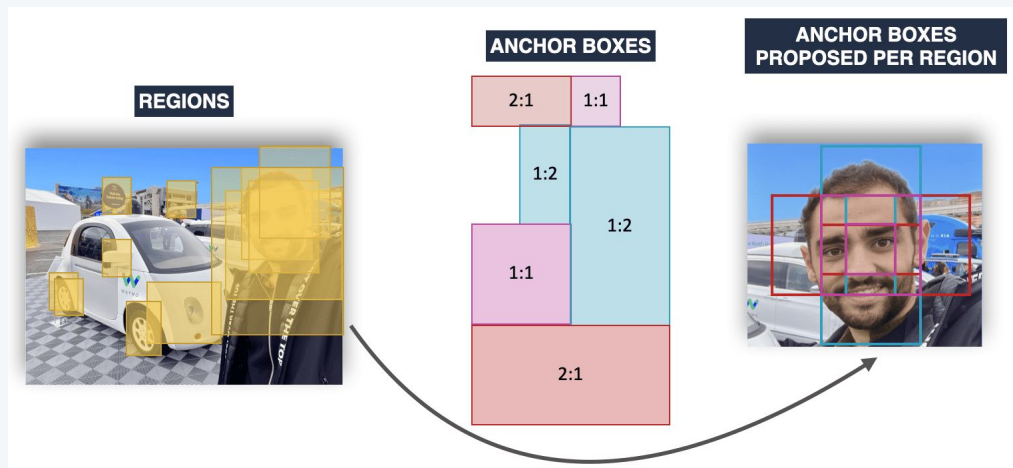
La salida de un modelo puede resultar en **varias respuestas** válidas.

Este algoritmo permite **eliminar duplicados** basándose en el valor de IoU entre dos o más bounding boxes.



Las detecciones no optimizan el espacio de detección, solo muestran la **zona más probable** en la que se encuentra un objeto.

Otra estrategia es fijar bounding boxes que sirvan como **anchor** sobre el que aplicar transformaciones pequeñas y **ajustar** la delimitación de las cajas.



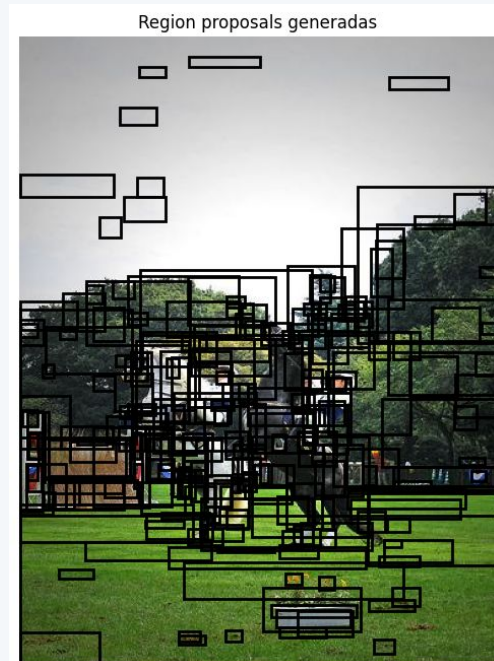
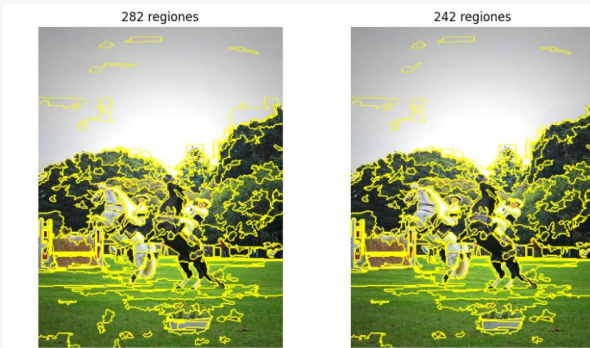
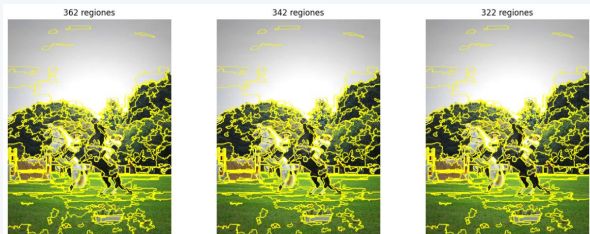
# Region-based CNN (R-CNN)

12

Otra aproximación al problema es detectar primero **regiones de interés** y luego aplicar las CNN.

Para detectar estas regiones se usa un algoritmo llamado **selective search**.

El resultado son detecciones más **ajustadas** al objeto.



Este algoritmo tiene los siguientes pasos:

1. Sobresegmentación: se **agrupan** conjuntos de píxeles similares en **color y textura**.
2. Búsqueda: se calcula la **similitud** entre regiones.
3. Fusión: se **unifican** las regiones más similares.

Tras esto, se **recortan** las regiones de interés y se **aplican CNN** para clasificarlas.

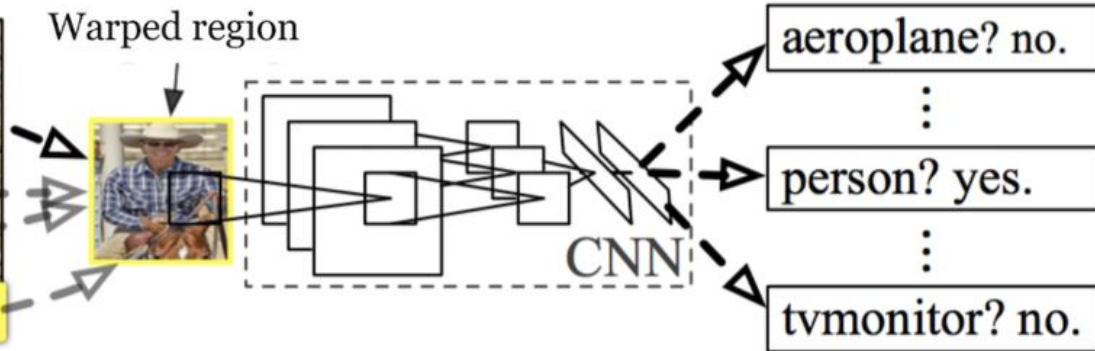




1. Input images



2. Extract region proposals (~2k)



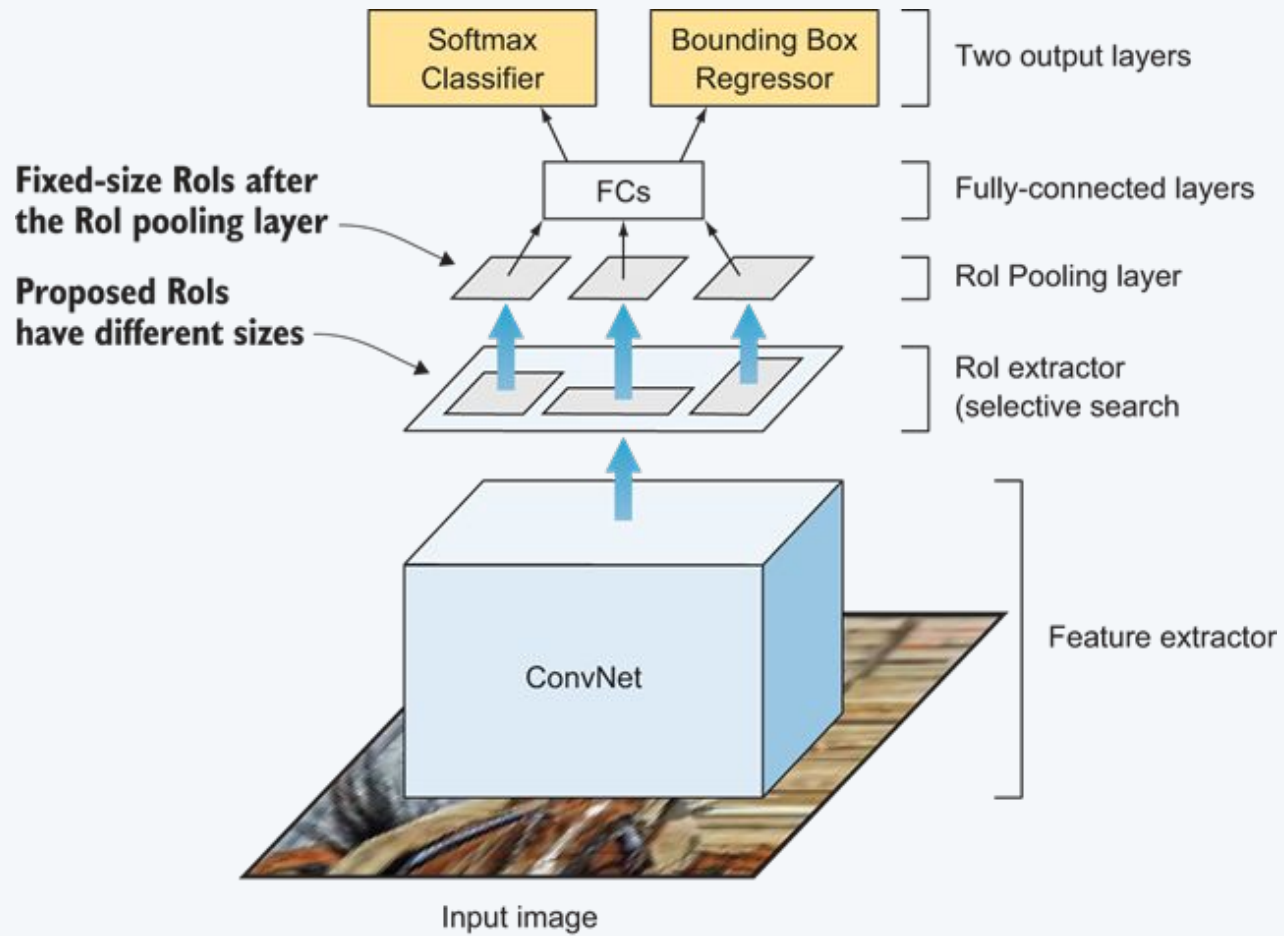
3. Compute CNN features

4. Classify regions

Para evitar procesamientos innecesarios se aplica nuevamente la técnica de generar un **feature map** para la imagen y aplicar un **clasificador de presencia** sobre estos.

La convolución permite **trasladar** la detección del selective search y encontrar más rápidamente las regiones candidatas ya **acotadas**.

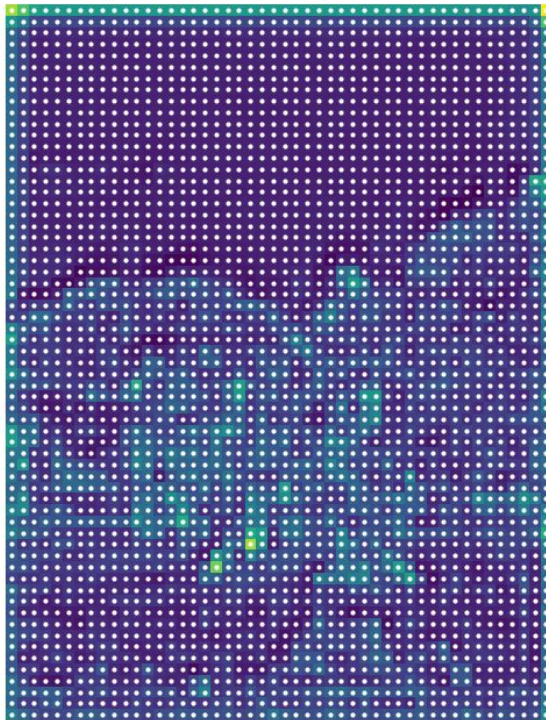




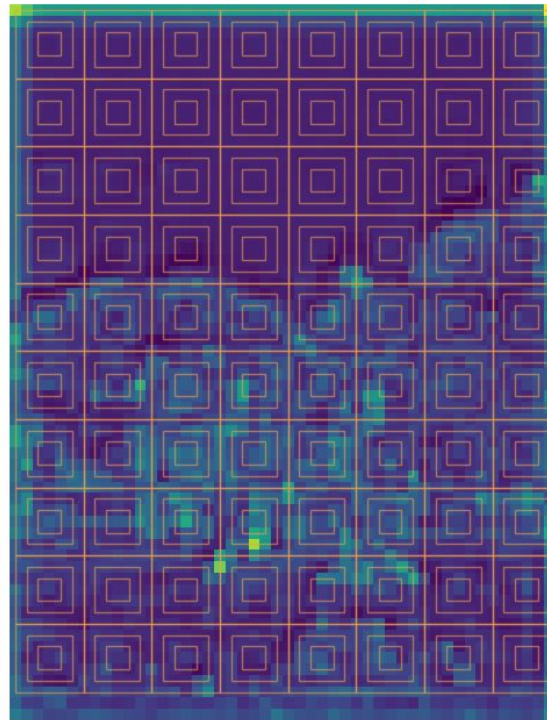
Selective search puede ser ineficiente y dar **muchas respuestas**. Este modelo reemplaza al algoritmo por una red que **paraleliza** funciones:

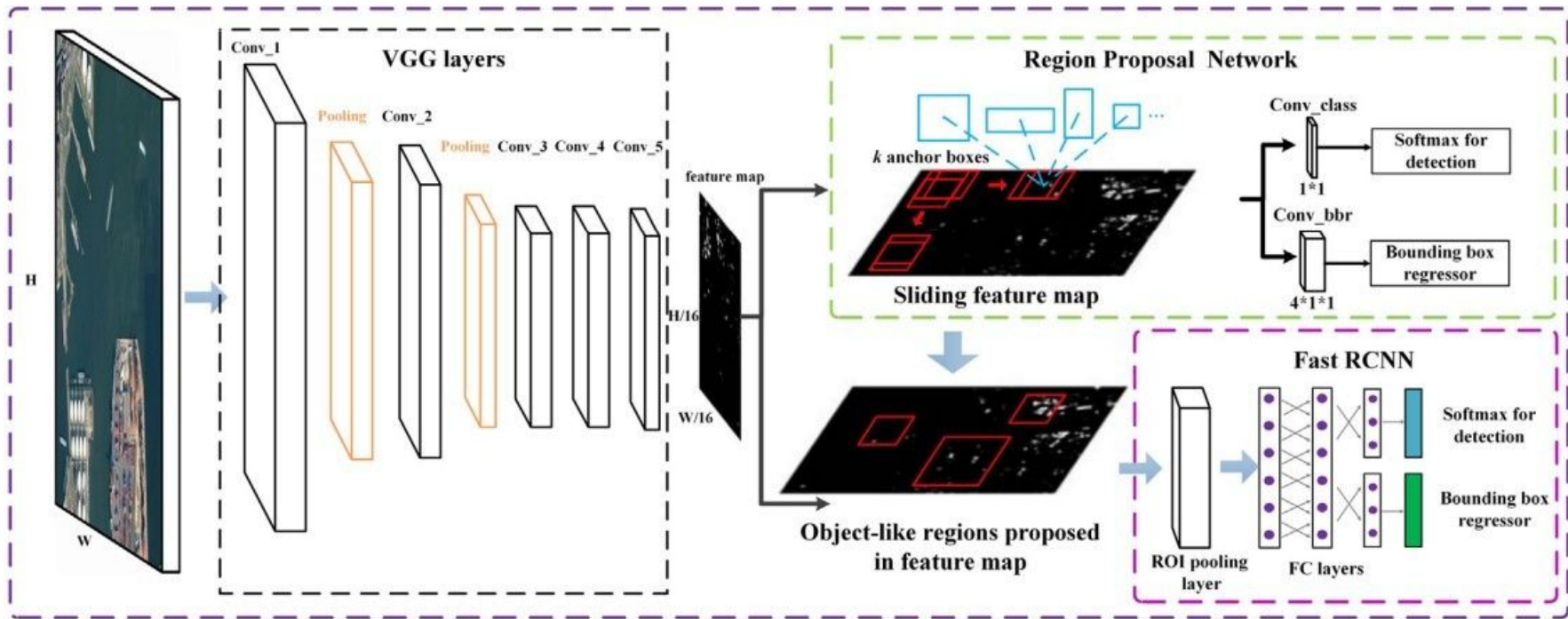
1. **Detecta** si una región del mapa de features es o no un objeto.
2. Propone **dimensiones** de la región.

Cada punto representa una posición de la RPN



Anchors sobre el Feature Map





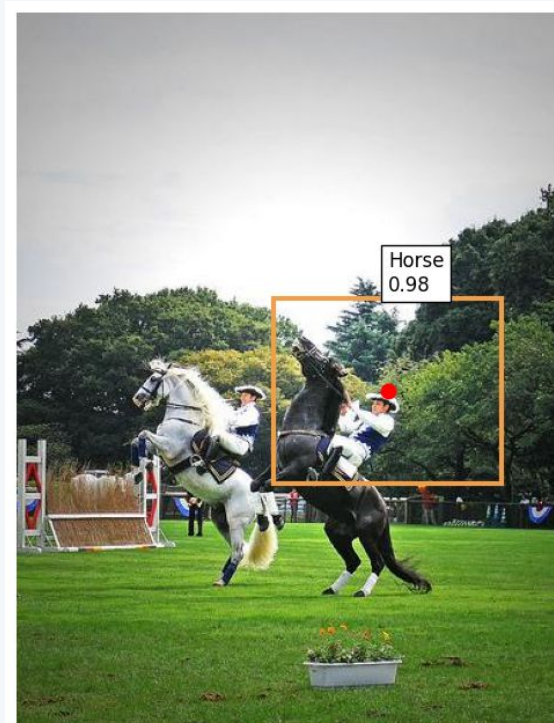
| Método           | Ejecución de CNN | ¿Reconocimiento previo? | Velocidad | Limitación                             |
|------------------|------------------|-------------------------|-----------|----------------------------------------|
| Sliding Window   | Miles de veces   | No                      | Muy baja  | Muchas ventanas                        |
| Convolutional SW | Una vez          | No                      | Media     | Cantidad de ventanas según convolución |
| R-CNN            | Miles de veces   | Sí (Alg. externo)       | Muy baja  | CNN para cada región                   |
| Fast R-CNN       | Una vez          | Sí (Alg. externo)       | Alta      | Cuello de botella en selective search  |
| Faster R-CNN     | Una vez          | Sí (Red neuronal)       | Muy alta  | Arquitectura compleja                  |

# You Only Look Once (YOLO)

20

Para acelerar más el proceso se busca hacer el paso de la presencia de objetos y su delimitación **a la vez**.

Para ello se busca una **codificación** de la información que permita detectar una posición y estimar la pertenencia a una clase **en simultáneo**.



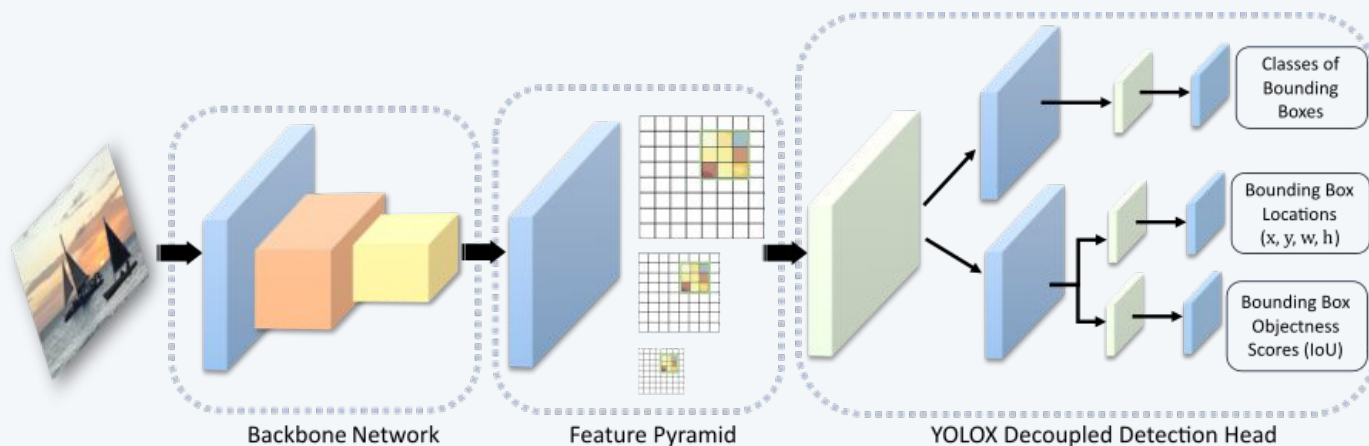
El formato YOLO crea un vector que contiene:

- La probabilidad de que en una celda haya un objeto de interés.
- La posición  $x$  del bounding box que enmarca el objeto.
- La posición  $y$  del bounding box que enmarca el objeto.
- El ancho del bounding box.
- El alto del bounding box.
- La probabilidad de pertenencia del objeto a cada clase del modelo.



Sus capas más profundas, llamadas **detection heads**, codifican la información de cada celda de un feature map con este formato.

Actualizar esta parte de la red permite predecir con precisión tanto la **posición** como la **categoría** de los objetos presentes en la imagen.

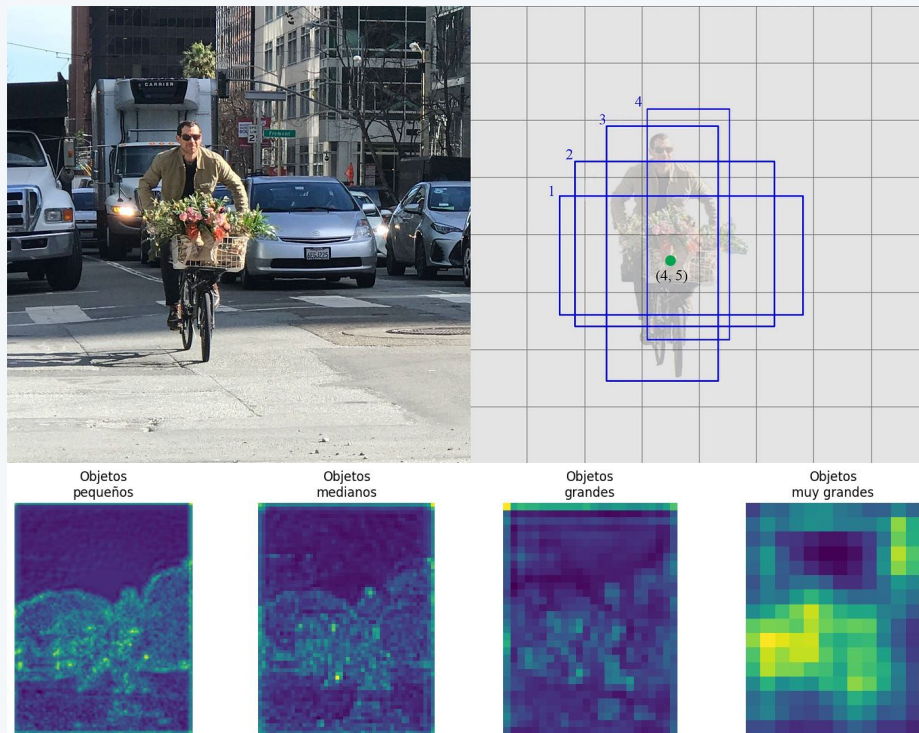


# Single Shot Multibox Detector (SSD)

23

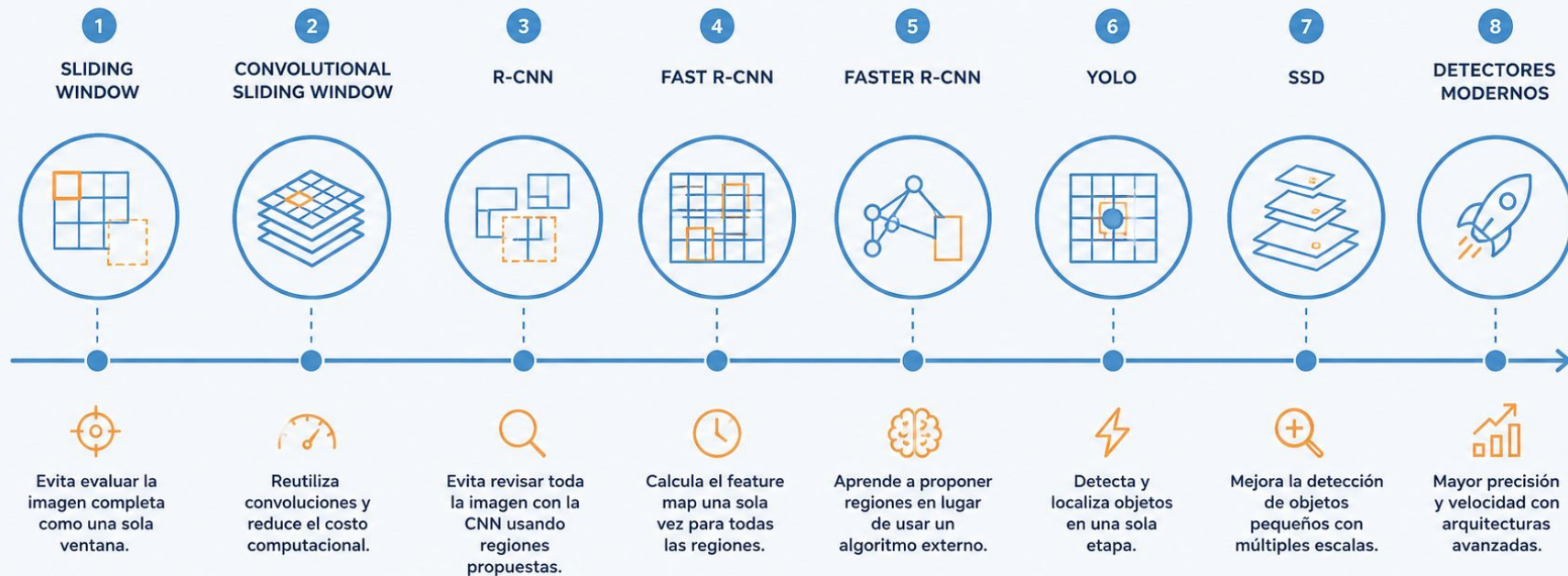
YOLO resultó ineficiente en caso de objetos muy **pequeños** o **cercanos**.

La arquitectura SSD plantea incorporar un **detector** sobre las capas **menos profundas** del feature map para poder detectar mayor detalle.



# Evolución de los métodos de detección

24

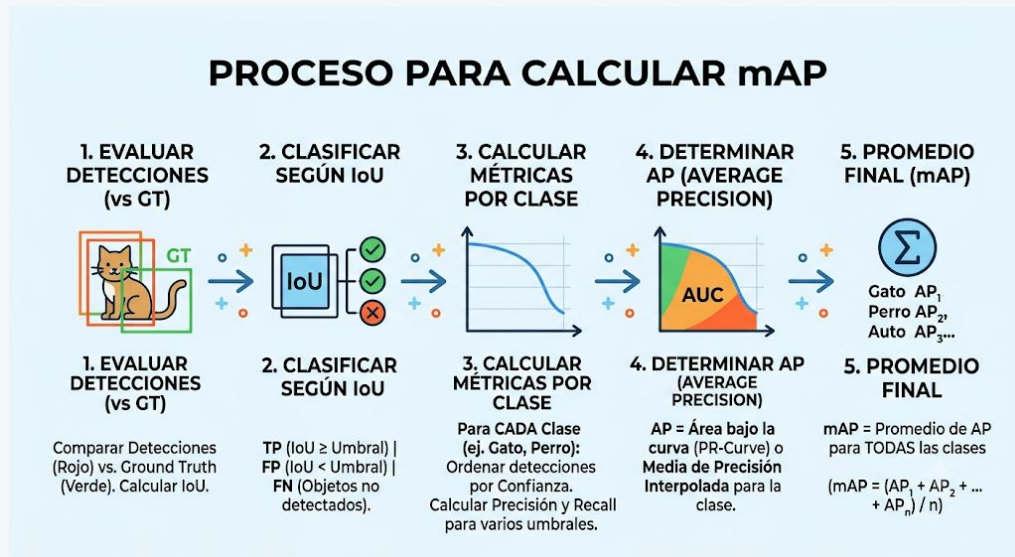


# Evaluación de modelos

25

La evaluación de este tipo de modelos debe considerar, además de que la **clase** predicha sea correcta, que la **ubicación** del bounding box sea correcta.

La métrica utilizada para medir la eficiencia es el **mean Average Precision** o **mAP**.



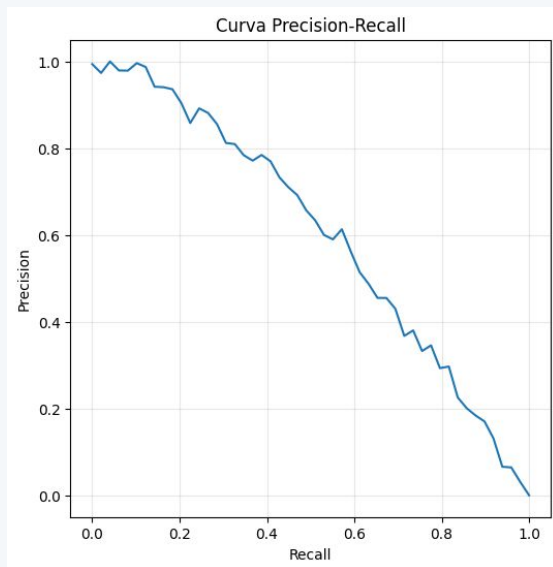
Primero mide el **IoU** del bounding box predicho con el etiquetado; y se filtran los que no **superan un umbral**.

Múltiples predicciones para un mismo objeto



Del conjunto restante, se ordenan las predicciones de mayor a menor **confianza** y se mide la **precisión y recall** de los objetos variando el umbral de confianza.

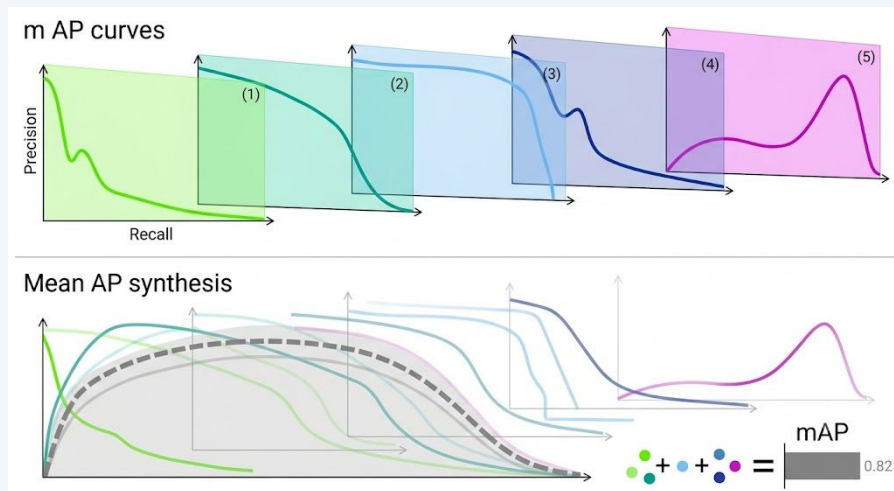
Esto genera una **curva característica** para los objetos de cada clase.



Se observa que si la precisión se mantiene constante mientras el recall aumenta, **el desempeño es mejor**.

Para medirlo se grafica la **curva precision-recall** y se calcula su **área**; mientras mayor sea, mejor es el desempeño. Este valor es el **Average Precision** o **AP**.

Este proceso se repite para las **m** **clases** y se **promedia** por dicho valor para obtener el mAP.



**¿Alguna pregunta?**